# SIMATS ENGINEERING

## CO2_AT2 Reverse Engineering Task

**COURSE NAME: Software Testing** **COURSE CODE: CSA3712**

**COURSE OUTCOME COVERED**

**CO2:** *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

| Sl. No | Questions | Bloom's Level |
|---|---|---|
| 1 | A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation. | 5 |
| 2 | A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation. | 5 |
| 3 | A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation. | 5 |

| | | |
|---|---|---|
| 4 | A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic. | 5 |
| 5 | You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability. | 6 |
| 6 | A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram. | 5 |
| 7 | An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities. | 5 |
| 8 | A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors. | 5 |

| | | |
|---|---|---|
| 9 | A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss. | 5 |
| 10 | You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model. | 5 |

***Code of Conduct:***

*I Peta Harsha Vardhan (192311039) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***



## RUBRICS FOR CALCULATION:3

| Criteria | Weightage (%) | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Understanding of System | 20% | Demonstrates clear and in-depth understanding of the system/product and its functionality | Shows good understanding with minor gaps | Partial understanding; misses key aspects | Lacks understanding of the system/product |
| Decomposition | 25% | Effectively breaks down the system into components with detailed analysis | Decomposes system with minor gaps in analysis | Limited or incomplete decomposition | Unable to analyze or break down system |
| Identification of Components | 20% | Accurately identifies all components and explains their functions clearly | Identifies most components with minor errors | Identifies few components; explanations are weak | Unable to identify components or functions |
| Reconstruction | 20% | Successfully reconstructs or models the system with accuracy and logic | Reconstruction is mostly correct with minor issues | Partial reconstruction; lacks clarity | Unable to reconstruct or model system |
| Documentation | 15% | Well-organized, clear documentation with diagrams and structured explanation | Documentation is adequate with minor issues | Poorly organized or incomplete documentation | No proper documentation or presentation |

# Question 1

## Concept Involved

Reverse engineering of black-box test design – decomposition of a system into functional components, inference of equivalence classes and boundary conditions from observed behaviour, evaluation of test adequacy and reconstruction of test cases for an ATM withdrawal module.

## Definition

**Reverse engineering** starts from the observed behaviour of a system and works backwards to find the logic behind it and the test strategy that produced the observed tests. In black-box testing only inputs and outputs are visible, so the hidden rules (limits, validations, error messages) are inferred from the results. **Equivalence Class Partitioning (ECP)** groups inputs that the system treats alike, and **Boundary Value Analysis (BVA)** tests the edges of those groups, where defects are most likely.

## Scenario / Given Data

Only the following partial test log of the ATM withdrawal module is available (assumed for the analysis). Nothing else is known about the code.

| Obs. | Amount (₹) | Balance (₹) | Output | Message shown |
|---|---|---|---|---|
| O1 | 500 | 10,000 | Success | Please collect cash |
| O2 | 5,000 | 10,000 | Success | Please collect cash |
| O3 | 0 | 10,000 | Error | Invalid amount |
| O4 | 25,000 | 50,000 | Error | Exceeds per-transaction limit |
| O5 | 5,000 | 2,000 | Error | Insufficient balance |
| O6 | 550 | 10,000 | Error | Enter multiples of 100 |

## Answer (Step-by-Step Solution)

### Step 1: Decompose the system into functional components

The messages in the log show that the module applies a chain of checks, each producing its own error. The inferred decomposition is:

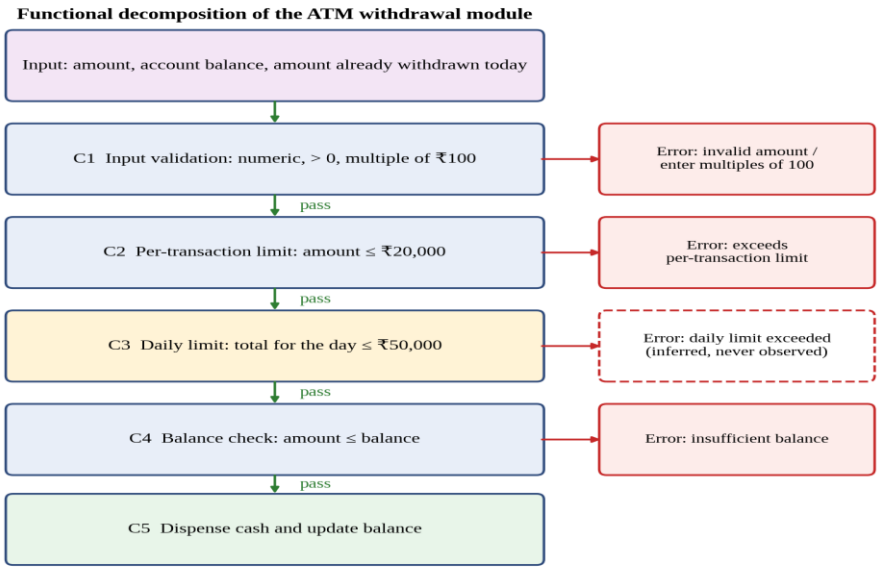



*Fig. 1: Inferred functional components of the ATM withdrawal module*

Component C3 (daily limit) is shown with a dashed box because no observation in the log exercises it; it is inferred from normal ATM behaviour and must be confirmed with the specification.

**Step 2: Infer the equivalence classes and boundary conditions**

| Class | Condition | Observed in log? | Evidence / remark |
|---|---|---|---|
| K1 | Valid: multiple of 100, within limits, ≤ balance | Yes | O1, O2 |
| K2 | Amount = 0 | Yes | O3 |
| K3 | Negative amount | No | Never tried |
| K4 | Non-numeric or empty input | No | Never tried |
| K5 | Not a multiple of 100 | Yes | O6 (550) |
| K6 | Above per-transaction maximum | Yes | O4 (25,000); limit inferred as ₹20,000 |
| K7 | Amount above balance | Yes | O5 |
| K8 | Above daily limit (₹50,000, inferred) | No | Never tried |

**Boundary conditions inferred (10 values):** minimum (99, 100, 200), maximum (19,900, 20,000, 20,100), balance (amount = balance, amount = balance + 100) and daily limit (total exactly at the limit, total just above). The values ₹20,000 and ₹50,000 are inferred; only the failure of ₹25,000 is observed, so the real limit may be any value below 25,000.

**Step 3: Infer the black-box techniques used and evaluate their adequacy**

| Observation | Inferred technique | Verdict |
|---|---|---|
| **One typical value per rule (500, 5,000, 25,000, 550)** | Equivalence class partitioning, one representative per class | Used, but incomplete |
| **Each error message is triggered once** | Requirement-based (functional) testing | Used |
| **No value at 99, 100, 20,000 or 20,100** | Boundary value analysis | Not used |
| **No test with daily total or negative or text input** | Negative testing and decision-table testing | Not used |

| Adequacy measure | Observed log (6 tests) | Reconstructed suite (16 tests) |
|---|---|---|
| **Equivalence classes covered** | 5 / 8 = 62.5 % | 8 / 8 = 100 % |
| **Boundary values tested** | 0 / 10 = 0 % | 10 / 10 = 100 % |
| **Negative / invalid input classes** | 2 of 4 (zero, non-multiple) | 4 of 4 |
| **Check-order and combined rules** | Not examined | Examined (R15, R16) |

**Evaluation:** the original design is functional and class-based, but it is inadequate because it never touches a boundary, which is where withdrawal modules typically fail (for example > 20,000 coded in place of ≥), and it leaves three of the eight classes untried.

**Step 4: Reconstructed test cases covering all critical scenarios**

| ID | Amount (₹) | Balance (₹) | Withdrawn today (₹) | Expected result | Covers |
|---|---|---|---|---|---|
| R01 | 500 | 10,000 | 0 | Success | K1 |
| R02 | 0 | 10,000 | 0 | Invalid amount | K2 |
| R03 | −100 | 10,000 | 0 | Invalid amount | K3 |

| ID | Amount (₹) | Balance (₹) | Withdrawn today (₹) | Expected result | Covers |
|---|---|---|---|---|---|
| R04 | “abc” | 10,000 | 0 | Invalid input | K4 |
| R05 | 550 | 10,000 | 0 | Enter multiples of 100 | K5 |
| R06 | 99 | 10,000 | 0 | Error (below minimum) | BVA min− |
| R07 | 100 | 10,000 | 0 | Success | BVA min |
| R08 | 200 | 10,000 | 0 | Success | BVA min+ |
| R09 | 19,900 | 50,000 | 0 | Success | BVA max− |
| R10 | 20,000 | 50,000 | 0 | Success | BVA max |
| R11 | 20,100 | 50,000 | 0 | Exceeds per-transaction limit | K6, BVA max+ |
| R12 | 5,000 | 5,000 | 0 | Success, balance becomes 0 | BVA balance = |
| R13 | 5,100 | 5,000 | 0 | Insufficient balance | K7, BVA balance+ |
| R14 | 5,000 | 10,000 | 45,000 | Success (daily total 50,000) | BVA daily = |
| R15 | 100 | 10,000 | 50,000 | Daily limit exceeded | K8, BVA daily+ |
| R16 | 20,000 | 15,000 | 0 | Insufficient balance | Check order |

All eight classes and all ten boundary values are covered by 16 tests.

**Step 5: Documentation of the reasoning**

- Every reconstructed test is traceable to a class (K1 – K8) or a boundary value, so any later failure can be linked to a specific rule.
- Inferred limits (₹20,000 and ₹50,000) are marked as assumptions; if the specification gives different values, only the test data change, not the design.
- R16 checks that a valid amount fails only on the balance rule, which confirms the order of the checks C1 → C4.

## Application

Reverse engineering of test design is used when testing legacy banking, UPI and ATM software that has no documentation, when auditing the quality of a vendor’s test suite, and when preparing regression tests for a system that is being migrated.

## Conclusion

The observed log shows class-based, functional testing with one typical value per rule and no boundary testing, covering only 62.5 % of the equivalence classes and none of the boundary values. By decomposing the module into five components, inferring eight classes and ten boundary values, and reconstructing 16 test cases, complete and traceable coverage of the ATM withdrawal logic is achieved.

## Question 2

### Concept Involved

Reverse engineering of white-box testing – control flow graph (CFG) construction from code, decision points and loops, independent (basis) paths, cyclomatic complexity, coverage criteria and path testing.

### Definition

When only code and execution traces are available, the white-box test approach is recovered by drawing the **CFG**, listing which nodes and edges the traces executed, and measuring the coverage reached. **Statement coverage** counts executed statements; **branch coverage** counts decision outcomes taken; **path coverage** counts execution paths. The **cyclomatic complexity** V(G) = E − N + 2 = number of decisions + 1 gives the number of independent paths that must be tested.

### Scenario / Given Data

The module code and two execution traces are given (assumed for the analysis); there is no test documentation.

| Node | Statement |
|---|---|
| N1 | total = 0;  i = 1 |
| N2 (D1) | while (i <= n) |
| N3 (D2) | if (price[i] > 0) |
| N4 (D3) | if (member) |
| N5 | total = total + price[i] × 0.9 |
| N6 | total = total + price[i] |
| N7 | i = i + 1 |
| N8 (D4) | if (total > 1000) |
| N9 | total = total − 50 |
| N10 | return total |

| Trace | Input | Executed nodes (result) |
|---|---|---|
| T1 | n = 2, price = [200, 300], member = TRUE | N1, N2, N3, N4, N5, N7 (twice), N2, N8, N10 → 450 |
| T2 | n = 1, price = [100], member = FALSE | N1, N2, N3, N4, N6, N7, N2, N8, N10 → 100 |

**Answer (Step-by-Step Solution)**

**Step 1: Construct the control flow graph and identify decisions and loops**

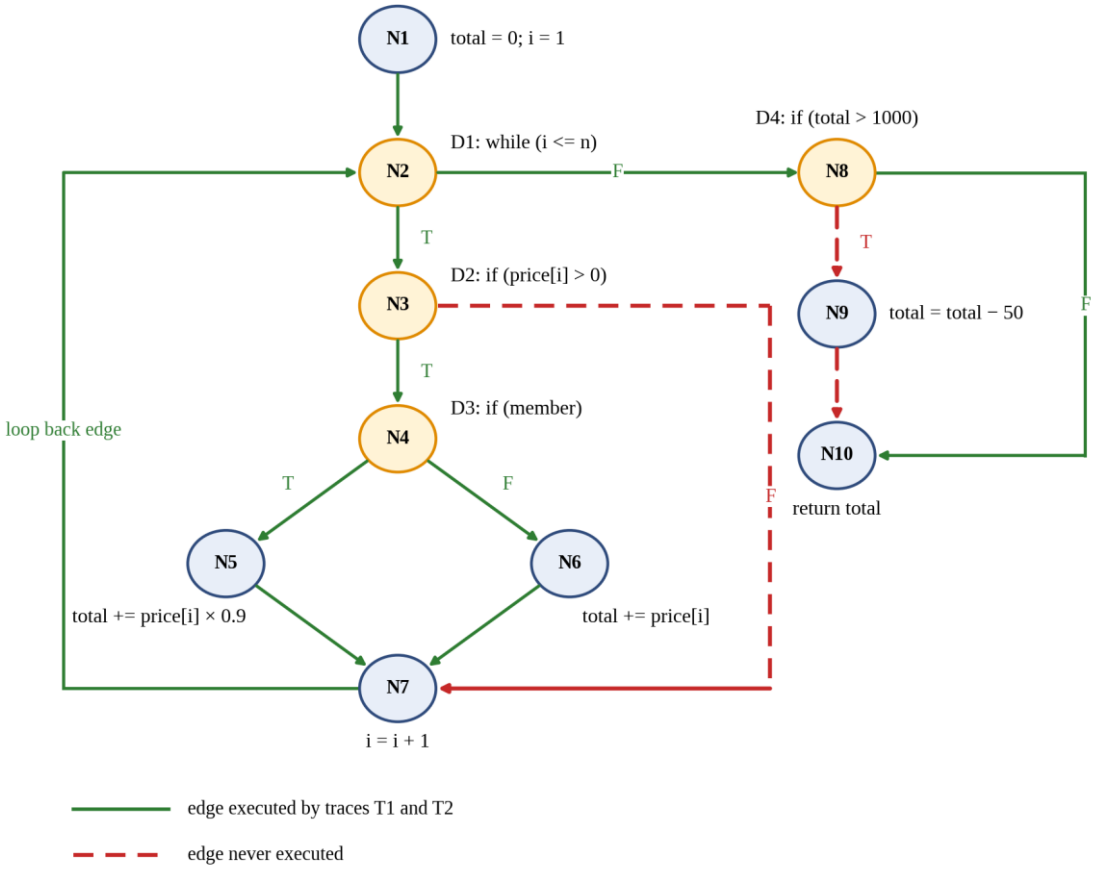



*Fig. 2: CFG of the module – green: edges executed by T1 and T2, red: edges never executed*

- **Decision points:** D1 (loop condition), D2 (price > 0), D3 (member), D4 (total > 1000).
- **Loop:** one while loop, N2 → N3 … N7 → N2.
- **Cyclomatic complexity:** nodes N = 10, edges E = 13, so V(G) = 13 − 10 + 2 = **5**, which equals the number of decisions (4) + 1.

**Step 2: Evaluate the existing test adequacy**

| Criterion | Observation | Result |
|---|---|---|
| **Statement coverage** | Only N9 (total − 50) never executed | 9 / 10 = 90 % |
| **Branch coverage** | D2 false, D4 true never taken (D1 and D3 both ways covered) | 5 / 8 = 62.5 % |
| **Independent paths** | T1 and T2 exercise only two of the five paths | 2 / 5 = 40 % |
| **Loop coverage** | Only n = 1 and n = 2 iterations | 2 of 7 cases |

**Reverse-engineered approach:** the two traces look like "typical scenario" tests (one member, one non-member) written without a coverage target. They reach high statement coverage but ignore the false branch of D2, the discount threshold D4 and the loop boundaries.

**Step 3: Independent paths (basis set)**

| Path | Node sequence | Decisions | Test data | Expected |
|---|---|---|---|---|
| P1 | N1–N2–N8–N10 | n = 0, total ≤ 1000 | n = 0 | 0 |
| P2 | N1–N2–N3–N4–N5–N7–N2–N8–N10 | price > 0, member | n = 1, [200], TRUE | 180 |
| P3 | N1–N2–N3–N4–N6–N7–N2–N8–N10 | price > 0, non-member | n = 1, [200], FALSE | 200 |
| P4 | N1–N2–N3–N7–N2–N8–N10 | price ≤ 0 | n = 1, [−5] | 0 |
| P5 | N1–N2–N3–N4–N6–N7–N2–N8–N9–N10 | total > 1000 | n = 1, [1200], FALSE | 1150 |

*The path "n = 0 and total > 1000" is infeasible because total is 0 when the loop is skipped, so it is excluded.*

**Step 4: Reconstructed path testing strategy**

1. Run the five basis-path tests P1 – P5 (each introduces at least one new edge).
2. Add boundary tests for the two data conditions D2 (price = 0, 1) and D4 (total = 1000, 1001).
3. Add loop tests: skip, one pass, two passes, typical, maximum − 1, maximum and maximum + 1 passes (maximum n = 50 assumed).
4. Measure coverage with a tool and add a test for any remaining uncovered edge.

| ID | Test data | Purpose | Expected |
|---|---|---|---|
| B1 | n = 1, [1000], FALSE | D4 boundary: total = 1000, no deduction | 1000 |
| B2 | n = 1, [1001], FALSE | D4 boundary: total = 1001 | 951 |
| B3 | n = 1, [0] | D2 boundary: price = 0 | 0 |
| B4 | n = 1, [1], FALSE | D2 boundary: price = 1 | 1 |
| B5 | n = 1, [1200], TRUE | Member with total above 1000 | 1030 |
| L1 | n = 2, [200, 300], TRUE | Two passes (trace T1) | 450 |
| L2 | n = 5, [100, 200, 0, 300, 400], FALSE | Typical passes; total exactly 1000 | 1000 |
| L3 | n = 49, every price 10, FALSE | Maximum − 1 passes | 490 |
| L4 | n = 50, every price 10, FALSE | Maximum passes | 500 |
| L5 | n = 51 | Maximum + 1 (rejected or handled) | Error |

In total the strategy uses 15 test cases (P1 – P5, B1 – B5, L1 – L5).

**Step 5: Coverage achieved after reconstruction**

| Criterion | Before (T1, T2) | After (15 tests) |
|---|---|---|
| **Statement coverage** | 90 % | 100 % |
| **Branch coverage** | 62.5 % | 100 % |
| **Independent paths (V(G) = 5)** | 2 / 5 | 5 / 5 |
| **Loop cases** | 2 / 7 | 7 / 7 |

**Reasoning:** V(G) = 5 is the minimum number of tests for full independent-path coverage; the boundary and loop tests add the cases that path coverage alone does not reveal, such as a total of exactly 1000.

## Application

This reverse-engineering approach is used to audit legacy code that has no test documentation, to estimate the quality of an inherited test suite, and to create regression tests before modifying billing, payroll or inventory modules.

## Conclusion

From the code and the two traces, the CFG (N = 10, E = 13, V(G) = 5) shows that the existing tests achieved 90 % statement coverage but only 62.5 % branch coverage and 40 % path coverage. The reconstructed strategy of five basis-path tests, five boundary tests and five loop tests raises every criterion to 100 % and exposes the untested discount threshold and loop boundaries.

## Question 3

### Concept Involved

Reverse engineering of a testing approach from defect logs – defect pattern analysis, identifying the black-box and white-box techniques used, system decomposition, coverage gap evaluation and reconstruction of an integrated strategy (ECP, BVA, path testing).

### Definition

Defects that **escape** testing reveal which techniques were missing, while defects that were **caught** reveal which techniques were applied. For example, boundary defects found only in production point to the absence of **boundary value analysis**, and an unexecuted branch points to the absence of **path or branch coverage**. The reconstructed strategy combines **ECP**, **BVA** and **path testing** so that each technique covers the weakness of the others.

### Scenario / Given Data

Only the defect log of a web login-and-transaction application is available (assumed for the analysis):

| ID | Defect | Found in | Pattern |
|---|---|---|---|
| DL1 | No error message after a wrong password | System test | Typical invalid class |
| DL2 | Transfer larger than the balance accepted | System test | Typical invalid class |
| DL3 | Password of exactly 8 characters rejected (valid range 8 – 16) | Production | Boundary value |
| DL4 | Account not locked after the 3rd wrong attempt (locked at 4th) | Production | Boundary value |
| DL5 | Transfer equal to the full balance rejected | Production | Boundary value |
| DL6 | Expired session token accepted for a transfer | Production | Unexecuted branch |
| DL7 | User name containing a quote (') causes a server error | Production | Invalid / malicious input |
| DL8 | Double-click on “Pay” debits the account twice | Production | Concurrency |

### Answer (Step-by-Step Solution)

### Step 1: Analyse the defect patterns and infer the techniques used

| Technique | Evidence from the defect log | Verdict |
|---|---|---|
| **Equivalence class partitioning** | Typical wrong-password and overdraft defects (DL1, DL2) were caught in system test | Used, for typical classes only |
| **Boundary value analysis** | Three boundary defects (DL3, DL4, DL5) escaped | Not used |
| **Branch / path testing** | The token-expiry branch was never executed (DL6) | Not used |
| **Error guessing / security testing** | Quote character and rapid repeated clicks not tried (DL7, DL8) | Not used |

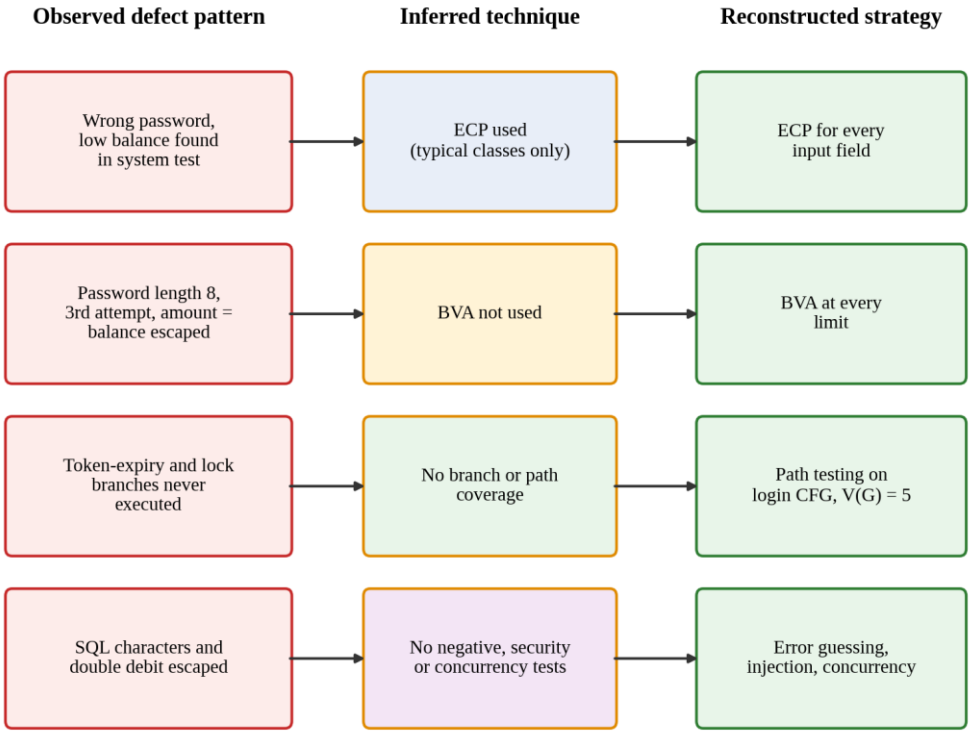



*Fig. 3: From defect patterns to inferred techniques and the reconstructed strategy*

**Conclusion of the inference:** the project relied on black-box functional testing with typical values (ECP) and did not use boundary, structural or negative testing.

### Step 2: Decompose the system into testable components

| Component | Functions | Gap evaluated from the log |
|---|---|---|
| **C1 Login** | User and password validation, failed-attempt counter, account lock | Boundaries of length and attempts untested |
| **C2 Transaction** | Amount validation, balance check, debit/credit, confirmation | Balance boundary and duplicate request untested |
| **C3 Session and security** | Session token validity, input sanitisation | Token-expiry branch and special characters untested |

### Step 3: Reconstructed strategy – ECP and BVA

| Input | Equivalence classes | Boundary values to test |
|---|---|---|
| **Password length** | < 8 (invalid), 8 – 16 (valid), > 16 (invalid) | 7, 8, 9, 15, 16, 17 |
| **Failed attempts** | 1 – 2 (no lock), ≥ 3 (lock) | 2nd and 3rd failed attempt |
| **Transfer amount** | ≤ 0 (invalid), 1 – balance (valid), > balance (invalid) | 0, 1, balance − 1, balance, balance + 1 |
| **User name** | Valid text, special characters | Quote, semicolon, empty |

**Step 4: Reconstructed strategy – path testing of the login logic**

| Node | Login logic (inferred) |
|---|---|
| N1 | read(user, password) |
| D1 | user exists? – No → reject (unknown user) |
| D2 | password length 8 – 16? – No → reject (length) |
| D3 | password correct? – Yes → create session |
| D4 | failed attempts ≥ 3? – Yes → lock account; No → reject (wrong password) |

Four decisions give V(G) = 4 + 1 = **5** independent paths: P1 unknown user, P2 bad length, P3 wrong password (attempts < 3), P4 wrong password with lock (attempts ≥ 3), P5 successful login. The transaction code adds the decision "session token valid?" (DL6), which needs its own path test.

**Step 5: Reconstructed test suite**

| ID | Component | Input / condition | Technique | Expected result |
|---|---|---|---|---|
| L01 | Login | Unknown user name | Path P1 | Reject |
| L02 | Login | Password of 7 characters | BVA, P2 | Reject (length) |
| L03 | Login | Correct password of 8 characters | BVA, P5 | Login success |
| L04 | Login | Correct password of 9 characters | BVA | Login success |
| L05 | Login | Correct password of 15 characters | BVA | Login success |
| L06 | Login | Correct password of 16 characters | BVA | Login success |
| L07 | Login | Password of 17 characters | BVA, P2 | Reject (length) |
| L08 | Login | Wrong password, 1st attempt | ECP, P3 | Error message shown |
| L09 | Login | Wrong password, 2nd attempt | BVA, P3 | Error, not locked |
| L10 | Login | Wrong password, 3rd attempt | BVA, P4 | Account locked |
| L11 | Login | Correct password after lock | P4 | Reject (locked) |
| L12 | Login | Correct credentials, valid state | P5 | Session created |
| T01 | Transaction | Expired token, amount 100 | Branch test | Reject (session expired) |
| T02 | Transaction | Amount 0 | BVA | Reject |
| T03 | Transaction | Amount 1 | BVA | Success |
| T04 | Transaction | Amount = balance − 1 | BVA | Success |
| T05 | Transaction | Amount = balance | BVA | Success |
| T06 | Transaction | Amount = balance + 1 | BVA, ECP | Reject (insufficient) |
| T07 | Security | User name ' OR '1'='1 | Error guessing | Reject, no server error |
| T08 | Transaction | Two identical Pay requests at once | Concurrency | Debited once |

**Step 6: Justification**

| Defect | Caught by | Why |
|---|---|---|
| DL1 | L08 | Wrong-password error message verified on the first failed attempt |
| DL2 | T06 | Amount above balance is a boundary and an invalid class |
| DL3 | L03 | Length 8 is the lower boundary of the valid class |
| DL4 | L10 | The 3rd attempt is the lock boundary |
| DL5 | T05 | Amount equal to balance is the upper boundary |
| DL6 | T01 | Forces the token-expiry branch to execute |
| DL7 | T07 | Malicious input class added by error guessing |
| DL8 | T08 | Concurrency test with duplicate requests |

All eight logged defects are caught by at least one of the 20 tests, and the integrated design covers input classes (ECP), their edges (BVA) and the code structure (path testing).

**Application**

Defect-pattern analysis is used in post-release reviews, vendor test-quality audits and root-cause analysis of production incidents in banking, e-commerce and other web applications.

**Conclusion**

The defect log shows that only typical-class testing (ECP) was done: boundary, branch and negative scenarios escaped to production. Decomposing the system into login, transaction and security components and combining ECP, BVA and path testing (V(G) = 5 for login) gives a 20-test strategy that detects all eight logged defects and is logically justified by the defect patterns.

## Question 4

### Concept Involved

Reverse engineering business logic from transaction logs – hidden boundary conditions, equivalence partitions, functional decomposition and black-box test design for an inventory (stock update and low-stock alert) system.

### Definition

In a legacy system with no specification, the logs of successful and failed transactions act as examples of the rules. Comparing a pair of log entries that differ by one unit (for example a success and a failure) pinpoints a **boundary**, and entries that behave alike form an **equivalence partition**. The inferred logic is then used to design black-box tests that fill the gaps in the logs.

### Scenario / Given Data

The following stock-update log is available (assumed for the analysis):

| Log | Stock before | Operation | Quantity | Result |
|---|---|---|---|---|
| 1 | 100 | Issue | 30 | Success, stock 70 |
| 2 | 70 | Issue | 70 | Success, stock 0, STOCK-OUT alert |
| 3 | 0 | Issue | 1 | Failed: insufficient stock |
| 4 | 100 | Issue | 0 | Failed: invalid quantity |
| 5 | 100 | Issue | −5 | Failed: invalid quantity |
| 6 | 100 | Receive | 50 | Success, stock 150 |
| 7 | 990 | Receive | 20 | Failed: exceeds capacity |
| 8 | 100 | Issue | 85 | Success, stock 15, LOW-STOCK alert |
| 9 | 100 | Issue | 80 | Success, stock 20, LOW-STOCK alert |
| 10 | 100 | Issue | 79 | Success, stock 21, no alert |
| 11 | 980 | Receive | 20 | Success, stock 1,000 |

### Answer (Step-by-Step Solution)

### Step 1: Infer the hidden business logic and boundary conditions

| Rule | Inferred logic | Log evidence |
|---|---|---|
| **R1 Quantity** | Quantity must be a positive number (≥ 1) | Logs 4 and 5 fail; 1 – 85 succeed |
| **R2 Issue** | Issue allowed only if quantity ≤ stock | Log 2 (70 of 70) succeeds; log 3 (1 of 0) fails |
| **R3 Capacity** | Receive allowed only if stock + quantity ≤ 1,000 | Log 11 (1,000) succeeds; log 7 (1,010) fails |
| **R4 Low-stock alert** | Alert when stock after the update is ≤ 20 | Log 9 (20) alerts; log 10 (21) does not |
| **R5 Stock-out alert** | Special alert when stock after the update is 0 | Log 2 |

**Hidden boundaries found:** quantity 0 / 1, quantity = stock / stock + 1, alert level 20 / 21 and capacity 1,000 / 1,001. The capacity of 1,000 and the alert level of 20 are inferred from pairs of logs that differ only slightly.

### Step 2: Decompose the update process into functional units

| Unit | Function | Rules |
|---|---|---|
| **U1 Input validation** | Check the quantity | R1 |
| **U2 Availability / capacity check** | Issue: quantity ≤ stock; Receive: stock + quantity ≤ 1,000 | R2, R3 |
| **U3 Stock update** | Compute the new stock level | – |
| **U4 Alert unit** | Raise stock-out or low-stock alert after the update | R4, R5 |

### Step 3: Equivalence partitions of the stock level after a transaction

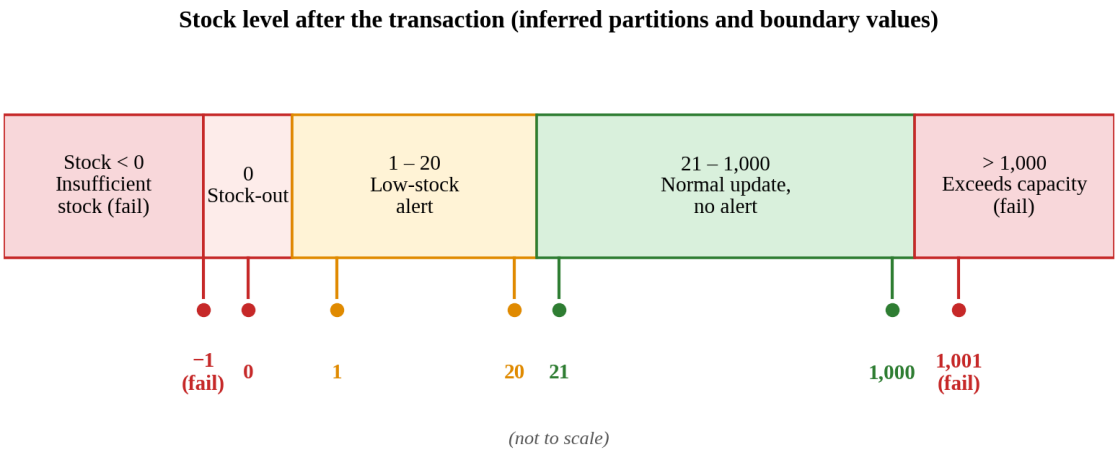



*Fig. 4: Inferred partitions and boundary values of the stock level after a transaction*

### Step 4: Gaps found in the logs

| Observation | Gap |
|---|---|
| **All 6 input partitions and all 3 stock levels appear in the log** | Partition coverage is good |
| **6 of the 8 boundary points appear** | Missing: quantity exactly 1 and stock + quantity exactly 1,001 |
| **Alert level only checked at 20 and 21 (and 15)** | New stock of 19 and 1 never checked |
| **Only integers appear** | Non-integer quantity untested |
| **Only "Issue" triggers alerts** | Alert clearing after a Receive, and recovery from stock-out, untested |
| **Receive with quantity ≤ 0 not logged** | Rule R1 for Receive untested |

### Step 5: Reconstructed black-box test design

| ID | Operation | Stock | Quantity | Expected result | Based on |
|---|---|---|---|---|---|
| BB1 | Issue | 100 | 1 | Success, stock 99, no alert | R1 boundary |
| BB2 | Issue | 100 | 0 | Invalid quantity | R1 boundary |
| BB3 | Issue | 100 | −1 | Invalid quantity | R1 |

| ID | Operation | Stock | Quantity | Expected result | Based on |
|---|---|---|---|---|---|
| BB4 | Issue | 100 | 2.5 | Rejected (to confirm) | Gap |
| BB5 | Issue | 100 | 79 | Stock 21, no alert | R4 boundary |
| BB6 | Issue | 100 | 80 | Stock 20, low-stock alert | R4 boundary |
| BB7 | Issue | 100 | 81 | Stock 19, low-stock alert | R4 |
| BB8 | Issue | 100 | 99 | Stock 1, low-stock alert | R4, R5 boundary |
| BB9 | Issue | 100 | 100 | Stock 0, stock-out alert | R2, R5 boundary |
| BB10 | Issue | 100 | 101 | Failed: insufficient stock | R2 boundary |
| BB11 | Receive | 100 | 50 | Stock 150 | R3 |
| BB12 | Receive | 980 | 20 | Stock 1,000 | R3 boundary |
| BB13 | Receive | 980 | 21 | Failed: exceeds capacity | R3 boundary |
| BB14 | Receive | 100 | 0 | Invalid quantity | R1 gap |
| BB15 | Receive | 10 (alert active) | 50 | Stock 60, alert cleared (to confirm) | R4 gap |
| BB16 | Receive | 0 | 100 | Stock 100, no alert | Stock-out recovery |

**Step 6: Justification of the selected tests**

- **Boundary tests (BB1, BB2, BB5, BB6, BB9, BB10, BB12, BB13)** cover all eight inferred boundary points, including the two missing from the logs.
- **Partition tests (BB3, BB7, BB11)** give one more representative for the quantity, alert and receive partitions.
- **Gap tests (BB4, BB8, BB14, BB15, BB16)** address the untested situations listed in Step 4.
- Tests marked “to confirm” depend on an assumption about the legacy rule and must be checked with the business owner; the other 14 follow directly from the inferred logic.

## Application

The approach is used when migrating legacy inventory, ERP, warehouse and retail systems, and when writing regression tests for software whose specification is lost.

## Conclusion

From eleven log entries the logic of the inventory update was inferred: positive quantities only, issue limited by stock, receive limited by a capacity of 1,000, and low-stock alert at 20 or below. The logs cover all partitions but only six of the eight boundary points, so a reconstructed set of 16 black-box tests with boundary, partition and gap cases gives complete coverage of the inferred business logic.

## Question 5

### Concept Involved

Reverse engineering of white-box requirements from an execution profile – mapping CPU and memory spikes to loops and decision clusters, conceptual CFG, basis path testing and stability under stress.

### Definition

An execution profile records CPU and memory use over time. A **flat high CPU** period usually corresponds to a long or nested loop, a **rising memory** curve to a loop that allocates objects without releasing them, and short bursts to decision clusters. By matching each period to a code structure, a conceptual **CFG** is drawn and **path testing** is aimed at the resource-intensive paths.

### Scenario / Given Data

Only the compiled library-management module and the profile of one stress test are available (profile values assumed for the analysis):

| Phase | Time (s) | CPU (%) | Memory (MB) | Inferred logic |
|---|---|---|---|---|
| A | 0 – 10 | 5 | 40 | Start-up and configuration |
| B | 10 – 25 | 35 → 60 | 40 → 60 | Catalogue search: one loop over books with a match decision |
| C | 25 – 40 | 95 (flat) | 60 (flat) | Overdue-fine processing: nested loops, members × loans, with decisions |
| D | 40 – 55 | 30 | 60 → 400 | Batch caching: loop that allocates a record per iteration |
| E | 55 – 60 | 10 | 400 (not released) | Idle but memory not freed: possible leak |

### Answer (Step-by-Step Solution)

**Step 1: Map the execution spikes to loops and decision clusters**

- **Phase B (CPU rises with data size):** a single loop whose work grows linearly with the number of books; a decision inside it (title matches?) decides whether a result is added.
- **Phase C (CPU 95 %, memory flat):** work grows with members × loans, so two nested loops are inferred; inside the inner loop a cluster of decisions (overdue? more than 30 days?) chooses the fine.
- **Phase D (memory climbs 60 → 400 MB):** each iteration of a batch loop adds an object to a cache, and the cache is not cleared.
- **Phase E (memory stays at 400 MB):** the allocated memory is never released, which suggests a missing cleanup path.

## Step 2: Conceptual control flow graph

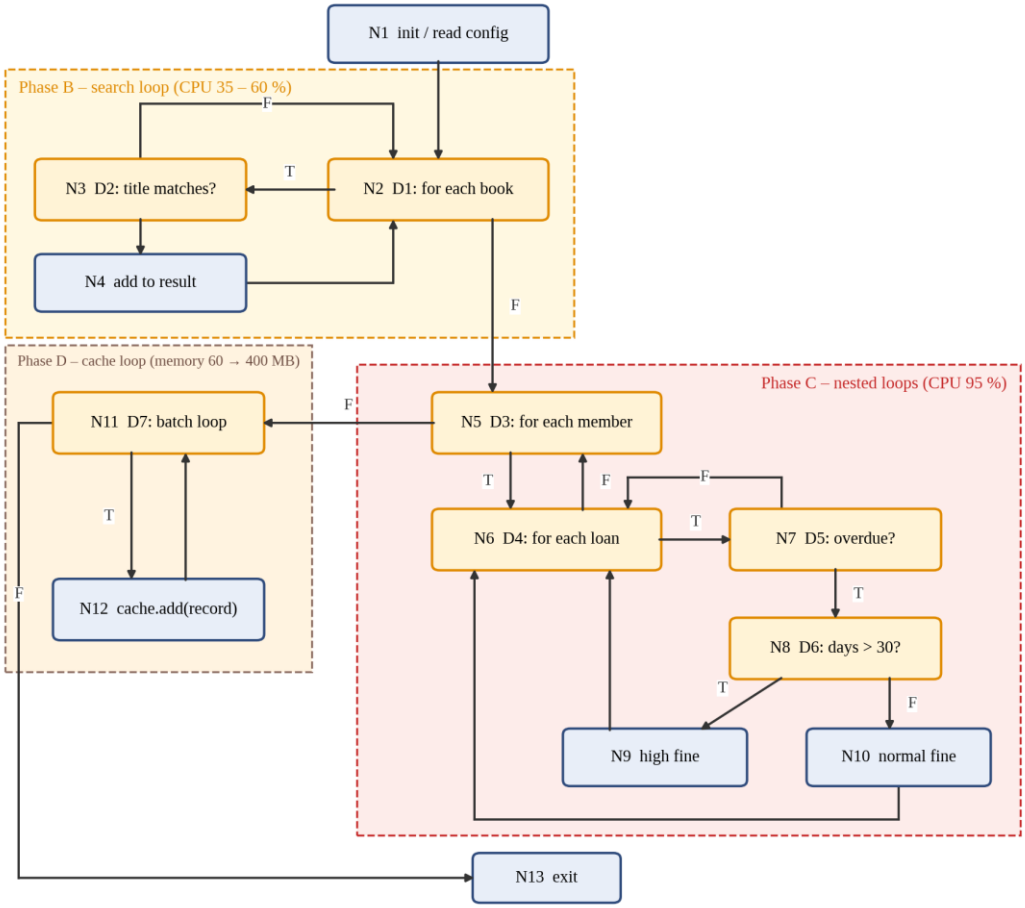



*Fig. 5: Conceptual CFG with the resource-intensive regions marked*

Nodes N = 13, edges E = 19, so V(G) = 19 − 13 + 2 = **8**, which equals the 7 decisions (D1 – D7) + 1. Eight independent paths are therefore needed.

## Step 3: Independent paths

| Path | Node sequence | Meaning | Resource cost |
|---|---|---|---|
| P1 | N1–N2–N5–N11–N13 | Empty library, nothing processed | Low |
| P2 | N1–N2–N3–N4–N2–N5–N11–N13 | Book matches the search | Low – medium |
| P3 | N1–N2–N3–N2–N5–N11–N13 | Book does not match | Low – medium |
| P4 | N1–N2–N5–N6–N5–N11–N13 | Member with no loans | Medium |
| P5 | N1–N2–N5–N6–N7–N6–N5–N11–N13 | Loan not overdue | High (nested) |
| P6 | N1–N2–N5–N6–N7–N8–N10–N6–N5–N11–N13 | Overdue 30 days or less | High (nested) |
| P7 | N1–N2–N5–N6–N7–N8–N9–N6–N5–N11–N13 | Overdue more than 30 days | High (nested) |
| P8 | N1–N2–N5–N11–N12–N11–N13 | Batch loop caches records | High (memory) |

**Step 4: Path testing strategy targeting the resource-intensive paths**

P5 – P7 sit inside the nested loops and P8 contains the allocation loop, so these paths are exercised with growing data sizes while CPU and memory are measured. The thresholds below are proposed targets, to be agreed with the project:

| ID | Path | Input | Measure | Target |
|---|---|---|---|---|
| S1 | P1 | 0 books, 0 members | CPU, memory baseline | CPU < 10 %, no error |
| S2 | P2 | 1 book that matches | Result correctness | Book returned |
| S3 | P3 | 1 book, no match | Result correctness | Empty result |
| S4 | P2, P3 | 1,00,000 books | Time versus size | Linear growth, CPU ≤ 60 % |
| S5 | P4 | 1,000 members, no loans | Outer loop only | CPU low |
| S6 | P5 | 1,000 members × 50 loans, none overdue | CPU, time | CPU ≤ 80 %, time ∝ members × loans |
| S7 | P6 | All loans overdue ≤ 30 days | CPU, fine value | CPU ≤ 80 %, fines correct |
| S8 | P7 | All loans overdue > 30 days | CPU, fine value | CPU ≤ 80 %, fines correct |
| S9 | P8 | One batch of 10,000 records | Memory before and after | Memory returns to baseline |
| S10 | P8 | 10 batches in a row | Memory trend | Flat, no growth per batch |

**Step 5: Impact on system stability**

| Aspect | Observed profile | Expected after the strategy |
|---|---|---|
| **CPU in the nested-loop phase** | 95 % flat | Kept within the 80 % target by limiting or optimising the member × loan scan |
| **Memory in the batch phase** | Rises to 400 MB, never released | Returns to baseline; cache cleared after each batch |
| **Path coverage** | Unknown | 8 / 8 independent paths executed |
| **Defect type found** | Not found by functional tests | Leaks and loop-bound problems found before release |

*These "expected" values are targets for the reconstructed strategy, not measured results.*

## Application

Profile-guided path testing is used for performance and stability testing of library, hospital, banking and e-commerce back-ends, for finding memory leaks, and for judging the risk of changing legacy code.

## Conclusion

The CPU and memory profile points to three code structures: a search loop, a nested member-and-loan loop with fine decisions, and a batch loop that caches records without cleanup. The conceptual CFG (N = 13, E = 19, V(G) = 8) gives eight independent paths; a stress-oriented path strategy of ten tests targets the high-cost paths P5 – P8, helps remove the 95 % CPU peak and the unreleased 400 MB, and improves stability.